Chapter 7 — GPU Kernels and CUDA Optimization

> *"A kernel is not just code that runs on a GPU. It is a contract between the algorithm and the hardware — specifying exactly how data moves through the memory hierarchy, how threads coordinate, and which hardware units do the work. Get the contract wrong, and you pay 10-100× in performance."*

Chapter Overview

This chapter builds the kernel-level understanding that every principal AI/ML performance architect needs. You do not need to write production CUDA kernels daily — but you must be able to read them, diagnose them with profiling tools, understand what makes them fast or slow, and know when a custom kernel is necessary versus when cuBLAS or Triton is the right choice. The profiling methodology here directly supports Chapters 12 and 17.

Sections in this chapter:

- 7.1 The GPU Execution Model — From Grid to Instruction
- 7.2 The GEMM Kernel — Tiling, SMEM, and Tensor Core Alignment
- 7.3 Memory Access Patterns — Coalescing, Bank Conflicts, Prefetch
- 7.4 Occupancy, Register Pressure, and the Efficiency Tradeoff
- 7.5 FlashAttention — IO-Aware Algorithm Design
- 7.6 Triton — Custom Kernel Development in Python
- 7.7 CUDA Graphs — Eliminating CPU Dispatch Overhead
- 7.8 Key LLM Kernels in Production
- 7.9 The Profiling Methodology — nsys → ncu → fix → verify
- 7.10 Key Takeaways and Review Questions

7.1 The GPU Execution Model — From Grid to Instruction

7.1.1 The Three-Level Thread Hierarchy

Every CUDA kernel launch specifies a three-level thread hierarchy that maps directly to GPU hardware. Understanding this mapping is the prerequisite for every optimization.

```
CUDA THREAD HIERARCHY:

  GRID (entire kernel launch)
  |  Represents: the complete parallel problem
  |  Dimensions: up to 2^31-1 blocks per dimension
  |  Scope: no direct communication across blocks
  |
  ├── BLOCK (Thread Block / CTA — Cooperative Thread Array)
  |   Represents: threads that share SMEM and can synchronize
  |   Max size: 1,024 threads per block (H100)
  |   Maps to: ONE SM (one SM can run multiple blocks if resources allow)
  |   Sync: __syncthreads() — barrier for all threads in block
  |
  |   ├── WARP (32 threads — the hardware execution unit)
  |   |   Represents: ACTUAL scheduling and execution granularity
  |   |   Size: always 32 threads (hardware constant since Fermi 2010)
  |   |   AMD equivalent: Wavefront = 64 threads
  |   |   Executes: SIMT — Single Instruction, Multiple Threads
  |   |   All 32 threads run the SAME instruction on DIFFERENT data
  |   |
  |   └── THREAD (individual execution stream)
  |         Has: up to 255 private 32-bit registers
  |         ID: threadIdx.x, threadIdx.y, threadIdx.z

HARDWARE MAPPING:
  Grid   -> GPU (all SMs)
  Block  -> SM (one SM, possibly multiple blocks if SMEM/regs allow)
  Warp   -> Warp scheduler (4 schedulers per SM on H100)
  Thread -> CUDA core or Tensor Core execution unit

H100 LIMITS: [SHIPPED]
  SMs per GPU:           132
  Max warps per SM:       64  (= 2,048 threads max)
  Max blocks per SM:      32
  Registers per SM:    65,536
  Shared Memory per SM: 228 KB (configurable)
  Max threads per block: 1,024
```

7.1.2 The SM Execution Engine — What Happens Each Cycle

```
H100 SM INSTRUCTION PIPELINE:
```

```
  4 × Warp Schedulers (each issues 2 instructions/cycle = 8 total/cycle)

  Per scheduler cycle:
    1. Select a READY warp from the active warp pool
    2. Issue its next instruction
    3. If warp stalls (e.g., HBM load): mark PENDING, switch to another ready warp
    4. When stall resolves: mark warp READY again

  This is the key GPU trick: hide memory latency by switching warps.
  H100 HBM latency: ~400 cycles. With 64 active warps: can keep schedulers
  busy executing other warps during the 400-cycle wait.

  Required occupancy for full latency hiding:
    Stall cycles = 400 (HBM)
    Instructions per warp between stalls = ~10 (typical)
    Warps needed = ceil(400/10) = 40 warps minimum = 62.5% occupancy

  This explains why ≥50% occupancy is the latency-hiding threshold.

TENSOR CORE THROUGHPUT:
  BF16 Tensor Core:  1,024 BF16 FMAs/cycle/SM
  FP32 CUDA core:      128 FP32 ops/cycle/SM
  Ratio: 8× → Tensor Cores are why AI is economically viable on GPUs

  To use Tensor Cores:
    Data type must be BF16, FP16, FP8, INT8, or TF32
    Matrix dimensions must align to TC tile sizes (16 for BF16, 128 for FP8)
    Operands must be in SMEM or registers in WGMMA-compatible layout
```

7.1.3 Warp Divergence — The Branch Tax

```
WARP DIVERGENCE:

  All 32 threads in a warp share one program counter.
  When different threads need different code paths:
    GPU serializes: execute true-branch (false threads masked),
    then false-branch (true threads masked).
    Cost: up to 2× execution time for 50/50 split.

  // DIVERGENT: Both branches run for ALL 32 threads
  if (threadIdx.x < 16) {
      result = compute_A();    // threads 0-15 active, 16-31 masked
  } else {
```

```
        result = compute_B();    // threads 16-31 active, 0-15 masked
    }
    // Cost: 2× a single branch. 50% utilization per branch.

    // NON-DIVERGENT: All 32 threads take the same path
    if (blockIdx.x < N/2) {
        result = compute_A();    // entire warp follows one branch
    } else {
        result = compute_B();
    }
    // Cost: 1× a single branch. Full utilization.

LLM KERNEL DIVERGENCE SCENARIOS:
    Causal attention mask: upper-triangle is zero → divergence per element.
    Fix: FlashAttention tiles — apply mask to entire tile at once (uniform).

    Variable sequence lengths: different batch elements have different lengths.
    Fix: Pack sequences to equal length OR process per-sample with separate kernels.

    Top-k/top-p sampling: conditional logic across token positions.
    Fix: Use reduction kernels (warp-uniform comparisons) rather than per-thread
branches.

DETECTING WARP DIVERGENCE IN PROFILER:
    Nsight Compute: smsp__average_warp_latency_per_inst_issued
    High value = warps frequently stalling (can indicate divergence).
    More direct: ncu --metrics
smsp__sass_average_data_bytes_per_wavefront_mem_shared
    Or: check "Warp State Statistics" tab — look for "Pred Off" cycles.
```

7.2 The GEMM Kernel — Tiling, SMEM, and Tensor Core Alignment

GEMM (C = A × B) is the most important operation in any AI model. Understanding its optimization from first principles enables you to diagnose any GEMM kernel.

7.2.1 Why Naive GEMM Is Catastrophically Slow

```
NAIVE GEMM (no tiling):

    // C[M×N] = A[M×K] × B[K×N]
    for i in range(M):
        for j in range(N):
```

```
        for k in range(K):
            C[i,j] += A[i,k] * B[k,j]

  EVERY multiply-accumulate requires loading A[i,k] and B[k,j] from HBM.
  Total HBM reads: 2 × M × N × K element loads

  For M=N=K=4096 (typical LLM weight matrix), BF16:
    Reads: 2 × 4096^3 = 137 billion element loads
    Bytes: 137 × 10^9 × 2 = 274 TB of HBM reads
    At H100 peak (3.35 TB/s): 274 TB / 3.35 TB/s = 81.7 seconds
    Reality: this is completely bandwidth-bound

  Arithmetic intensity of naive GEMM:
    FLOPs = 2 × M × N × K = 137 GFLOPs
    Bytes = 2 × M × N × K × 2B = 274 GB
    AI = 137 GFLOPs / 274 GB = 0.5 FLOP/byte
    Far below H100 ridge point (295) → 590× below optimal!
```

7.2.2 Tiled GEMM — How cuBLAS Achieves Near-Peak Performance

```
TILED GEMM WITH SMEM (CUTLASS pattern):

  Idea: load a tile of A and B into SMEM, compute all outputs using those tiles.
  This dramatically reduces HBM accesses.

  // Config: TILE_M=128, TILE_N=256, TILE_K=32
  for tile_i in range(0, M, TILE_M):
      for tile_j in range(0, N, TILE_N):
          // Accumulator in registers
          C_tile[TILE_M][TILE_N] = 0  (in registers)

          for tile_k in range(0, K, TILE_K):
              // Load ONE tile of A from HBM → SMEM
              A_smem[TILE_M][TILE_K] = A[tile_i:tile_i+TILE_M,
tile_k:tile_k+TILE_K]
              // Load ONE tile of B from HBM → SMEM
              B_smem[TILE_K][TILE_N] = B[tile_k:tile_k+TILE_K,
tile_j:tile_j+TILE_N]

              // Compute TILE_M × TILE_N × TILE_K inner products FROM SMEM
              // (All SMEM reads — fast, ~80 TB/s vs HBM's 3.35 TB/s)
              C_tile += A_smem × B_smem   // wgmma instruction
```

```
        // Write result tile to HBM (once per tile)
        C[tile_i:..., tile_j:...] = C_tile

ARITHMETIC INTENSITY WITH TILING:
  FLOPs per A-tile load: TILE_M × TILE_N × TILE_K × 2 = 2 × 128 × 256 × 32 = 2.1M
  Bytes per A-tile load: TILE_M × TILE_K × 2B = 128 × 32 × 2 = 8 KB
  AI per tile operation: 2.1M / 8K = 262 FLOP/byte

  For TILE_N=256: AI = 2 × 128 × 256 × 32 / ((128×32 + 32×256) × 2)
                     = 2,097,152 / (8,192 + 16,384) × 2 = ~85 FLOP/byte
  At TILE_M=TILE_N=128, TILE_K=32: AI ≈ 128 FLOP/byte
  At TILE_M=128, TILE_N=256, TILE_K=64: AI ≈ 225 FLOP/byte → approaching ridge!

SMEM REQUIREMENT:
  A_smem: TILE_M × TILE_K × 2B = 128 × 64 × 2 = 16 KB
  B_smem: TILE_K × TILE_N × 2B = 64 × 256 × 2 = 32 KB
  C_smem: optional — often kept in registers
  Total: 48 KB per block (within 228 KB limit — allows ~4 blocks per SM)
```

7.2.3 Tensor Core Alignment Requirements

```
TENSOR CORE ALIGNMENT — THE SILENT PERFORMANCE KILLER:

H100 WGMMA instruction requirements (BF16):
  M dimension: multiple of 64 (warpgroup size)
  N dimension: multiple of 256 (wgmma output tile width)
  K dimension: multiple of 16

Common alignment failures in LLM inference:
  Vocabulary projection: [d_model × vocab_size] = [8192 × 128,256]
    128,256 / 16 = 8,016  (divisible by 16 ✓)
    128,256 / 256 = 501   (NOT divisible by 256 ×)
    Fix: pad vocabulary to 128,512 (next multiple of 256)

  Custom head counts: H_kv=7 (not power of 2) → misaligned
    Fix: pad to H_kv=8 with dummy heads (masked in attention)

PERFORMANCE IMPACT OF MISALIGNMENT:
  ALIGNED:    M=N=K=4096 (all divisible by 256) → 850+ TFLOPS (H100)
  MISALIGNED: M=N=K=4097 → partial TC tiles → 30-40% of peak
  The fallback: partial tiles execute on CUDA cores (128 FP32 ops/cycle)
  vs Tensor Cores (1,024 BF16 FMAs/cycle) = 8× slowdown for partial tiles.
```

```
DIAGNOSTIC:
  ncu --metrics sm__pipe_tensor_cycles_active.avg.pct_of_peak
  Target: > 60% for GEMM kernels
  Low TC utilization at correct data type = alignment problem
```

7.2.4 Software Pipelining — Hiding Tile Load Latency

```
DOUBLE BUFFERING (SMEM SOFTWARE PIPELINE):

  Problem: Loading next SMEM tile from HBM takes ~30-50 clock cycles.
  During this load, Tensor Cores could be idle waiting for data.

  SINGLE BUFFER (naive):
    Step 1: Load tile A,B from HBM into SMEM  (wait ~40 cycles)
    Step 2: Compute C_tile += A_smem × B_smem  (compute ~20 cycles)
    Step 3: Load next tile...
    CUDA Core utilization: 20/(20+40) = 33%!

  DOUBLE BUFFER (pipelined):
    Buffer 0: holds tile K (being computed right now)
    Buffer 1: loading tile K+1 from HBM (async, overlapped with compute)

    async_load(buffer[1], A[K+1], B[K+1])  // issue async copy
    for each tile K:
        compute C_tile += A_smem[K%2] × B_smem[K%2]  // use buffer[K%2]
        __pipeline_wait_prior(1)                      // ensure next tile ready
        swap(K%2, (K+1)%2)                           // switch buffers

    GPU memory load latency completely hidden by Tensor Core compute.
    TC utilization: ~90%+ (limited by compute now, not HBM latency).

H100 WGMMA ASYNC (hardware-level pipelining):
  The wgmma instruction on H100 is ASYNCHRONOUS.
  SM issues wgmma → immediately issues next async SMEM load.
  Tensor Cores execute concurrently with data loading.
  No explicit double-buffer required — hardware handles it.
  This is a major H100 advance over A100's synchronous mma.sync.
```

7.3 Memory Access Patterns — Coalescing, Bank Conflicts, Prefetch

7.3.1 Global Memory Coalescing

```
COALESCED ACCESS — THE 32-ELEMENT RULE:

  When all 32 threads in a warp access consecutive memory addresses:
    GPU issues ONE 128-byte memory transaction.
    Efficiency: 32 elements loaded for cost of 1 transaction.
    Utilization: ~90% of peak HBM bandwidth.

  When threads access scattered addresses:
    GPU issues up to 32 separate transactions.
    Efficiency: 1 element per transaction (128-byte loads, 2-byte BF16).
    Utilization: ~3-7% of peak HBM bandwidth.

MATRIX LAYOUT AND COALESCING:

  Matrix A [M×K], row-major (standard C order, rows are contiguous):
    Thread j loads A[row][j] (all threads, same row, adjacent columns):
      → COALESCED (addresses differ by 2 bytes per thread)
    Thread j loads A[j][col] (all threads, different rows, same column):
      → UNCOALESCED (addresses differ by K×2 bytes per thread, stride=K)

  FIX FOR COLUMN ACCESS: transpose in SMEM
    Load tile from HBM into A_smem[TILE_K][TILE_M] (column-major in SMEM)
    This access pattern is coalesced (consecutive rows in HBM).
    Inside SMEM: now access by column → SMEM bank conflicts possible.
    Add padding: A_smem[TILE_K][TILE_M + 1] to break bank conflicts.

PRACTICAL COALESCING RULES:
  For a matrix [M×N] in row-major storage:
    Access pattern   | Coalesced? | HBM efficiency
    ──────────────────────────────────────────────────────────────
    A[i][tid]        | YES        | ~90%
    A[tid][j]        | NO (stride-N) | ~3-7%
    A[tid×S][j]      | NO (stride-S) | ~3-7% × min(S/32, 1)
    A[j×M+tid]       | YES (if j fixed) | ~90%
    ──────────────────────────────────────────────────────────────
```

7.3.2 Shared Memory Bank Conflicts

```
SMEM BANK STRUCTURE AND CONFLICTS:
```

```
  H100 SMEM: 32 banks, each 4 bytes wide, 32 banks × 4 bytes = 128-byte row.
  Bank for address addr: bank = (addr / 4) % 32

  // CONFLICT-FREE: Threads access different banks
  val = smem[threadIdx.x];          // thread i accesses bank i%32 → no conflict
  val = smem[threadIdx.x * 2];      // thread i accesses bank (2i)%32 → no
conflict
                                     // (power-of-2 strides avoid conflicts)

  // 2-WAY CONFLICT: Two threads access same bank
  val = smem[threadIdx.x * 17];     // thread 0→bank 0, thread 2→bank 2,
                                     // thread 32→bank 16, ...
                                     // periodic conflicts → 2× slowdown

  // 32-WAY CONFLICT: ALL threads access same bank
  val = smem[0];                     // all threads → bank 0 → 32 serialized
accesses
  val = smem[32];                    // all threads → bank 0 (32×4=128 → bank 0
again)
                                     // 32× slower than conflict-free!

  EXCEPTION (broadcast): if all threads in a warp access the EXACT same address,
  the hardware broadcasts the value → no conflict.

CANONICAL FIX — SMEM PADDING:
  // Original (conflicts for stride-32 access):
  __shared__ float smem[32][32];     // column access: all threads hit bank K%32

  // Fixed (add 1 padding column):
  __shared__ float smem[32][33];     // column K now at offset 33×4 bytes
                                     // bank = (i*33)%32 → strides differently
                                     // → no conflicts for column access

  Cost: ~3% SMEM waste. Worth it for any kernel doing column access.

DETECTION:
  ncu --metrics l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum
  Any non-zero value = bank conflict penalty.
  Target: < 1% conflict rate. Above 5% = likely fixable with padding.
```

7.4.1 The Three Occupancy Limiters

```
OCCUPANCY CALCULATION — THREE LIMITING RESOURCES:

  Definition: Active Warps per SM / Max Warps per SM (64 for H100)
  Each resource independently caps occupancy; the minimum wins.

  RESOURCE 1: REGISTER FILE
    H100: 65,536 registers per SM
    At reg/thread = R: max_warps = 65,536 / (R × 32)

    R=32:  max_warps = 65,536/1,024 = 64 → 100% occupancy
    R=64:  max_warps = 65,536/2,048 = 32 → 50% occupancy
    R=128: max_warps = 65,536/4,096 = 16 → 25% occupancy
    R=255: max_warps = 65,536/8,160 = 8  → 12.5% occupancy

  RESOURCE 2: SHARED MEMORY
    H100: 228 KB per SM
    At smem/block = S, block_size = B threads (B/32 warps):
    max_blocks = floor(228 KB / S)
    max_warps = max_blocks × (B/32)

    S=32 KB, B=256: max_blocks = 7, max_warps = 7×8 = 56 → 87.5%
    S=76 KB, B=256: max_blocks = 3, max_warps = 3×8 = 24 → 37.5%
    S=228 KB, B=256: max_blocks = 1, max_warps = 8 → 12.5% (FlashAttn)

  RESOURCE 3: BLOCK COUNT
    H100: max 32 blocks per SM
    max_warps = min(max_blocks, floor(smem_limit), ...) × warps_per_block

  FINAL OCCUPANCY = min(register_occupancy, smem_occupancy, block_occupancy)

OCCUPANCY ≠ PERFORMANCE:
  High occupancy does NOT guarantee high throughput.
  Low occupancy can still achieve peak throughput if:
    (a) The kernel is compute-bound (few memory stalls → few warps needed)
    (b) The kernel uses large tiles that fully saturate Tensor Cores

  FlashAttention v2: ~12-20% occupancy, achieves 55-75% of peak TFLOPS.
  Standard attention: ~80% occupancy, achieves 5-15% of peak TFLOPS.
  FlashAttention wins because it eliminates HBM traffic, not because of occupancy.
```

```
  RULE: Optimize for the bottleneck.
    Memory-bound kernel: high occupancy critical (hide 400-cycle stalls)
    Compute-bound kernel: moderate occupancy sufficient (TCs keep SMs busy)
```

7.4.2 Register Spilling — The Hidden Performance Killer

```
REGISTER SPILLING:

  When a kernel uses more registers than the hardware allocates:
    Compiler "spills" excess registers to local memory.
    Local memory maps to L2 cache (if hot) or HBM (if cold).

  Spilled register access: ~150-400 cycle latency (vs 1 cycle for register)
  This turns zero-latency computation into high-latency memory accesses.

  DETECTION:
    ncu --metrics l1tex__data_pipe_lsu_wavefronts_g_tex_l1_global_op_local.sum
    Any non-zero value = register spills to local memory.
    Also: nvcc compiler output: "-v --ptxas-options=-v" shows spill stats.

  MITIGATION:
    Option 1: Reduce register usage
      Use fewer temporary variables, recompute instead of storing.
      Use __launch_bounds__(MAX_THREADS, MIN_BLOCKS) to hint compiler.

    Option 2: Reduce block size
      Fewer threads per block → more registers available per thread.
      Trade: lower occupancy but fewer spills → net positive if compute-bound.

    Option 3: Accept spills and compensate with larger tiles
      If spills occur but arithmetic intensity is high enough:
        Tile size larger → more FLOPs between spill accesses → net positive.
      This is why FlashAttention v2 (200 registers/thread) still outperforms
      low-register-count naive attention (32 registers/thread).
```

7.5 FlashAttention — IO-Aware Algorithm Design

FlashAttention is the most important kernel innovation in the history of LLMs and the canonical example of hardware-aware algorithm design. Every principle in Chapters 4 and 7 converges here.

7.5.1 The Standard Attention HBM Bottleneck

```
STANDARD ATTENTION — O(S²) HBM TRAFFIC:

  For sequence S, head dim D, one attention head:

  Operation              HBM reads           HBM writes
  ──────────────────────────────────────────────────────────────
  Q[S×D], K[S×D] load    2 × S×D × 2B
  Compute S = QK^T
  Write S to HBM                             S×S × 4B (FP32)   ← O(S²)!
  Load S for softmax      S×S × 4B                              ← O(S²)!
  Softmax in-place
  Write softmax(S)                           S×S × 4B           ← O(S²)!
  Load softmax(S), V      S×S × 4B + S×D×2B                    ← O(S²)!
  Compute O = weights × V
  Write O                                    S×D × 2B
  ──────────────────────────────────────────────────────────────
  TOTAL:                 ≈ 4×S² × 4B + constant

  For LLaMA-3 70B, one head, S=4096, D=128, BF16:
    S² term: 4 × 4096² × 4 = 268 MB per head
    Total per layer (64 heads × 80 layers): 64 × 80 × 268 MB = 1.37 TB per pass

  FOR S=128K (long context):
    Per head per layer: 4 × 128000² × 4 = 262 GB — larger than H100 HBM!
    Standard attention literally cannot run at 128K context on H100.
```

7.5.2 FlashAttention — The O(S) Solution

```
FLASHATTENTION CORE INSIGHT:
  We do NOT need to materialize the full S×S attention matrix.
  We can compute the output incrementally, tile by tile, using SMEM only.
  The online softmax trick makes this mathematically exact.

TILED COMPUTATION PATTERN:

  Divide Q into row-tiles: Q_i of size [Br × D]
  Divide K, V into column-tiles: K_j, V_j of size [Bc × D]
  SMEM holds: Q_i + K_j + V_j + running accumulator (O_i, sum_i, max_i)

  For each query tile Q_i:
    Initialize: O_i = 0, sum_i = 0, max_i = -inf
```

```
    For each key-value tile (K_j, V_j):
      S_ij = Q_i × K_j^T             (compute in SMEM/registers)
      max_new = max(max_i, rowmax(S_ij))   (update running max)

      // Online softmax rescaling (key trick):
      O_i = O_i × exp(max_i - max_new)    // rescale previous accumulator
      sum_i = sum_i × exp(max_i - max_new) + rowsum(exp(S_ij - max_new))
      O_i = O_i + exp(S_ij - max_new) × V_j
      max_i = max_new

    // Normalize: divide by sum_i
    O_i = O_i / sum_i
    Write O_i to HBM  (once per Q tile)

HBM TRAFFIC — FLASHATTENTION:
  Reads:  Q once + K once + V once = 3 × S × D × 2B
  Writes: O once                   = 1 × S × D × 2B
  Total:  4 × S × D × 2B  =  O(S)  (not O(S²)!)

COMPARISON (LLaMA-3 70B, one head, S=4096, D=128):
  Standard attention:  268 MB per head per forward
  FlashAttention:      4 × 4096 × 128 × 2 = 4 MB per head per forward
  Reduction: 67×! At 64 heads × 80 layers: 1.37 TB → 20 GB.

SMEM SIZING FOR FLASHATTENTION:
  Must fit Q_i + K_j + V_j in SMEM simultaneously:
    Q_i: Br × D × 2B
    K_j: Bc × D × 2B
    V_j: Bc × D × 2B

  For H100 (228 KB SMEM), D=128, BF16:
    228,000 / (3 × 128 × 2) = 296 elements → Br = Bc = 64 (conservative)
    Or with careful packing: Br=128, Bc=64 (FlashAttention v2 default)

  This sets the occupancy at ~20% (large SMEM per block).
  But: HBM traffic reduced 67× → net throughput 3-5× better than standard.

FLASHATTENTION VERSIONS:
  v1 (2022): O(S) HBM traffic, no WGMMA async
  v2 (2023): Better parallelism over sequence dim, FP16/BF16 support, 2× v1
  v3 (2024, H100 only): WGMMA async + TMA, FP8 support, producer-consumer pipeline
    - 74% of H100 peak TFLOPS (vs v2's 55%)
```

```
    - 2× faster than v2 for long sequences
    - Already integrated into vLLM and TRT-LLM
```

7.5.3 The Online Softmax — Exact Computation Without Materialization

```
WHY ONLINE SOFTMAX IS MATHEMATICALLY EXACT:

  Standard softmax requires knowing ALL scores to compute denominator:
    softmax(x)_i = exp(x_i) / sum_j(exp(x_j))

  Numerically stable form:
    m = max(x)                              // global max, prevents overflow
    softmax(x)_i = exp(x_i - m) / sum_j(exp(x_j - m))

  Online version (process one chunk at a time):
    After chunk 1 (scores x₁):
      m₁ = max(x₁)
      s₁ = sum(exp(x₁ - m₁))
      o₁ = exp(x₁ - m₁) × V₁   // weighted value sum

    After chunk 2 (scores x₂, values V₂):
      m₂ = max(m₁, max(x₂))     // update global max estimate
      // Rescale previous results to new max:
      s₁ = s₁ × exp(m₁ - m₂)   // rescale old denominator
      o₁ = o₁ × exp(m₁ - m₂)   // rescale old accumulator
      // Add new chunk:
      s₂ = sum(exp(x₂ - m₂))
      o₂ = exp(x₂ - m₂) × V₂
      s_total = s₁ + s₂
      o_total = o₁ + o₂

    Normalize: output = o_total / s_total

  PROVEN EXACT: produces the same numerical result as materializing
  the full S×S matrix and applying standard stable softmax.
  This is the mathematical core of FlashAttention's correctness guarantee.
```

7.6 Triton — Custom Kernel Development in Python

Triton (OpenAI, 2019) is a domain-specific language for GPU kernels that provides near-CUDA performance from Python-like syntax. FlashAttention v2 is written in Triton.

7.6.1 Why Triton Exists and When to Use It

```
TRITON vs CUDA vs CUTLASS:

  CUDA C++:
    Pros: Full hardware control, absolute peak performance possible
    Cons: Verbose (~1,000 lines for a GEMM), requires deep architecture knowledge
          Register allocation, warp scheduling, memory hierarchy all manual
    Use: Final-stage optimization when Triton hits a ceiling

  CUTLASS / cuBLAS:
    Pros: NVIDIA-optimized GEMM and convolution, near-theoretical peak
    Cons: Only for GEMM/Conv, no custom fusion, limited flexibility
    Use: All standard GEMM operations (QKV projections, FFN weights)

  Triton Python:
    Pros: Python syntax, handles SMEM tiling/vectorization automatically
          Near-CUDA performance for memory-access-heavy patterns
          Fusion-first: designed to fuse operations across HBM boundaries
    Cons: Less control than CUDA, GEMM quality still below cuBLAS
    Use: Custom fused kernels (attention variants, custom activations,
         non-standard norms, custom quantization)

WHEN A CUSTOM KERNEL IS WORTH WRITING:
  1. A frequently-executed operation (>5% of step time) is memory-bound
     AND can be fused with adjacent operations to eliminate HBM round-trips.
  2. A new algorithm variant is needed that no library supports.
  3. The operation has non-standard memory access patterns (e.g., sparse).
  4. CUDA Graphs require fixed kernel structure; custom ops may break
compatibility.

WHEN TO USE EXISTING LIBRARIES:
  Dense GEMM (any shape): cuBLAS / CUTLASS — already near-optimal
  Standard FlashAttention: vLLM/TRT-LLM FAv3 — already at 75% peak
  Standard LayerNorm, RMSNorm: torch.compile fuses these automatically
  Standard SiLU/GELU: torch.compile handles fusion
```

7.6.2 Triton Programming Model — Key Concepts

```
TRITON PROGRAMMING MODEL:

  Triton programs over BLOCKS, not individual threads.
  The programmer specifies block sizes; Triton handles:
```

```
    - SMEM staging for block inputs
    - Thread-to-element mapping within the block
    - Vectorized memory access
    - Register allocation

  @triton.jit
  def vector_add(X_ptr, Y_ptr, Z_ptr, N, BLOCK_SIZE: tl.constexpr):
      # Block ID: which chunk this program handles
      pid = tl.program_id(axis=0)

      # Element range for this block
      block_start = pid * BLOCK_SIZE
      offsets = block_start + tl.arange(0, BLOCK_SIZE)  # [BLOCK_SIZE] indices

      # Mask for out-of-bounds (if N is not divisible by BLOCK_SIZE)
      mask = offsets < N

      # Load from HBM (coalesced — consecutive offsets)
      x = tl.load(X_ptr + offsets, mask=mask)
      y = tl.load(Y_ptr + offsets, mask=mask)

      # Compute in registers
      z = x + y

      # Store to HBM (coalesced)
      tl.store(Z_ptr + offsets, z, mask=mask)

  # Launch: ceil(N / BLOCK_SIZE) program instances
  grid = (triton.cdiv(N, BLOCK_SIZE),)
  vector_add[grid](X, Y, Z, N, BLOCK_SIZE=1024)

COMPILATION PIPELINE:
  Triton Python → LLVM IR → PTX (virtual GPU ISA) → SASS (hardware binary)
  First run: JIT compilation (~50-200ms overhead).
  Subsequent runs: cached binary (same SASS as CUDA).
```

7.6.3 Triton Autotuning — Finding Optimal Block Sizes

```python
```

```python
# AUTOTUNING: compiler benchmarks configs, selects fastest
@triton.autotune(
    configs=[
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 256, 'BLOCK_K': 64, 'num_stages':
        triton.Config({'BLOCK_M': 64,  'BLOCK_N': 256, 'BLOCK_K': 32, 'num_stages':
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 128, 'BLOCK_K': 32, 'num_stages':
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 64,  'BLOCK_K': 32, 'num_stages':
    ],
    key=['M', 'N', 'K'],   # retune when problem dimensions change
)
@triton.jit
def matmul_kernel(A_ptr, B_ptr, C_ptr, M, N, K,
                  BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexp
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    # Tile indices for this program
    rm = (pid_m * BLOCK_M + tl.arange(0, BLOCK_M))[:, None]  # [BLOCK_M, 1]
    rn = (pid_n * BLOCK_N + tl.arange(0, BLOCK_N))[None, :]  # [1, BLOCK_N]

    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)  # accumulator in regs

    for k in range(0, K, BLOCK_K):
        rk = k + tl.arange(0, BLOCK_K)
        A = tl.load(A_ptr + rm * K + rk[None, :])  # [BLOCK_M, BLOCK_K]
        B = tl.load(B_ptr + rk[:, None] * N + rn)  # [BLOCK_K, BLOCK_N]
        acc += tl.dot(A, B)                         # Tensor Core MMA

    C = acc.to(tl.bfloat16)
    tl.store(C_ptr + rm * N + rn, C)
```

7.6.4 Fused Kernel Example — RMSNorm + Linear Projection

```python
```

```python
@triton.jit
def fused_rmsnorm_linear_kernel(
    X_ptr,      # Input: [B, T, D]
    W_norm_ptr, # RMSNorm weight: [D]
    W_lin_ptr,  # Linear weight: [D, out_D]
    Y_ptr,      # Output: [B, T, out_D]
    B, T, D, out_D, eps,
    BLOCK_D: tl.constexpr,
    BLOCK_OUT: tl.constexpr,
):
    b = tl.program_id(0)   # batch index
    t = tl.program_id(1)   # token index

    row_offset = (b * T + t) * D
    offs_d = tl.arange(0, BLOCK_D)

    # STEP 1: Load X[b,t,:] from HBM — ONE HBM read
    x = tl.load(X_ptr + row_offset + offs_d, mask=offs_d < D)

    # STEP 2: RMSNorm — entirely in registers, no HBM write
    x_sq_sum = tl.sum(x * x)
    rms = tl.sqrt(x_sq_sum / D + eps)
    w_norm = tl.load(W_norm_ptr + offs_d, mask=offs_d < D)
    x_normed = (x / rms) * w_norm
    # NOTE: x_normed never written to HBM — stays in registers!

    # STEP 3: Linear projection — in registers
    offs_out = tl.arange(0, BLOCK_OUT)
    W = tl.load(W_lin_ptr + offs_d[:, None] * out_D + offs_out[None, :])
    y = tl.sum(x_normed[:, None] * W, axis=0)   # (D,) × (D, out_D) → (out_D,)

    # STEP 4: Write output to HBM — ONE HBM write
    out_offset = (b * T + t) * out_D
    tl.store(Y_ptr + out_offset + offs_out, y, mask=offs_out < out_D)

# HBM traffic: 1 read (X) + 1 read (W_norm) + 1 read (W_lin) + 1 write (Y)
# vs unfused:  read X, write normed_X, read normed_X, read W_lin, write Y
# Fusion eliminates 2 HBM round-trips → ~1.5-2× speedup
```

7.7 CUDA Graphs — Eliminating CPU Dispatch Overhead

7.7.1 The CPU Dispatch Problem

```
WITHOUT CUDA GRAPHS (Python → CUDA every step):

  For each decode step (one token generated):
    Python: model forward() call
      → PyTorch dispatcher: attribute lookup, argument checking
      → CUDA API: cuLaunchKernel (RMSNorm)        ~50-200 μs
      → CUDA API: cuLaunchKernel (QKV GEMM)       ~50-200 μs
      → CUDA API: cuLaunchKernel (FlashAttention) ~50-200 μs
      → CUDA API: cuLaunchKernel (FFN GEMMs ×3)   ~150-600 μs
      → ... (150+ kernel launches per transformer layer)
    GPU: actually executes (fast, ~5-50 ms total)

  CPU overhead per decode step: 2-10ms (Python + CUDA API dispatch)
  GPU compute per decode step:  15-50ms
  CPU overhead fraction: 2-10ms / (2-10ms + 15-50ms) = 4-40% overhead!

  At batch=1 (TPOT target = 30ms): 8ms CPU overhead = 27% of total latency.
  This is not a compute problem — it is an administration problem.

WITH CUDA GRAPHS:

  CAPTURE PHASE (once at startup, ~5-10 minutes):
    Run one decode step while GPU captures all CUDA API calls.
    Result: a compiled graph of all kernel launches in sequence.
    Memory: pre-allocated, addresses fixed at capture time.

  REPLAY PHASE (every subsequent decode step):
    Single call: cudaGraphLaunch(graph, stream)  →  ~10-100 μs overhead
    GPU executes all captured kernels identically.

  CPU overhead: 0.01-0.1ms (one API call) vs 2-10ms (many calls).
  Reduction: 20-100× lower dispatch overhead per step.

CUDA GRAPH REQUIREMENTS:
  1. Static graph structure: same kernels in same order every step.
     No dynamic control flow during the captured region.
  2. Fixed input shapes: buffer addresses captured at graph time.
     Variable-length inputs require: padding to fixed max length,
     or separate graphs per batch size (vLLM captures 20+ graph variants).
```

```
  3. Pre-allocated memory: all tensors allocated before capture.
     No cudaMalloc inside the graph region.

CUDA GRAPHS IN PRACTICE:
  vLLM: captures graphs at batch sizes [1,2,4,8,16,32,...,256]
        selects appropriate graph at runtime based on current batch.
  TRT-LLM: graphs captured at build time, single deployment graph.
  torch.compile: can generate CUDA Graph-compatible compiled code.

PERFORMANCE IMPACT:
  Inference serving (LLaMA-3 70B, H100, batch=1):
    Without CUDA Graphs: TPOT ~45ms (35ms GPU + 10ms CPU dispatch)
    With CUDA Graphs:    TPOT ~37ms (35ms GPU + 2ms overhead)
    Improvement: 17.8% TPOT reduction

  At batch=32 (TPOT ~25ms):
    Without: 25ms + 8ms = 33ms
    With:    25ms + 0.5ms = 25.5ms
    Improvement: 22.7% TPOT reduction
```

7.8 Key LLM Kernels in Production

7.8.1 The Kernel Map — What Runs in Each Forward Pass

```
LLM INFERENCE KERNEL MAP (LLaMA-3 style, one transformer layer):

  Kernel                | Phase    | Bottleneck          | % Time | Optimization

  QKV Projection GEMM   | Both     | Compute (prefill)   | 25%    | TC alignment,
FP8
                        |          | Bandwidth (decode)  |        | quantization
  FlashAttention v3     | Both     | Compute (prefill)   | 20%    | FAv3 for H100
                        |          | Bandwidth (decode)  |        | GQA kernel
  Output Proj GEMM      | Both     | Compute/BW          | 8%     | Same as QKV
  FFN Gate GEMM (W3)    | Both     | Compute/BW          | 12%    | Fuse with SiLU
  FFN Up GEMM (W1)      | Both     | Compute/BW          | 12%    | Fuse with gate
  FFN Down GEMM (W2)    | Both     | Compute/BW          | 12%    | Standard GEMM
  RMSNorm × 2           | Both     | Memory bandwidth    | <1%    | Fuse with GEMM
  SiLU Gate Activation  | Both     | Memory bandwidth    | <1%    | Fuse with W3
GEMM
```

```
  Residual Add × 2      | Both     | Memory bandwidth    | <1%    | Fuse with
RMSNorm
  Sampling (top-p/k)    | Decode   | Memory bandwidth    | <1%    | Separate
optimization
  AllReduce (TP)        | Both     | NVLink BW           | Var    | Overlap with
GEMM

──────────────────────────────────────────────────────────────────────────────

  KEY INSIGHT: ~85% of compute time is in GEMM kernels.
  Optimizing GEMM → improves everything.
  Optimizing LayerNorm in isolation → <1% total improvement.
  Always use the kernel map to prioritize optimization effort.

KERNEL OPTIMIZATION PRIORITY:
  1. Verify FlashAttention v3 is active for H100 (not v1 or v2)
  2. Verify FP8 is active via Transformer Engine (2× GEMM throughput)
  3. Enable CUDA Graphs for decode path (15-25% TPOT reduction)
  4. Verify RMSNorm is fused with adjacent GEMM (saves ~8% step time)
  5. For FP8: verify SiLU is fused with gate GEMM (2 GEMM loads → 1)
```

7.8.2 FlashAttention v3 — H100-Specific Optimizations

```
FLASH ATTENTION V3 INNOVATIONS (H100 SXM5):

  Problem with FAv2: designed for A100's synchronous mma.sync instruction.
  H100 adds: WGMMA (async warpgroup MMA) + TMA (Tensor Memory Accelerator).

  Innovation 1: WGMMA Producer-Consumer Pipeline
    Two warp groups per SM block:
      Producer: uses TMA to asynchronously load K, V tiles HBM → SMEM
      Consumer: uses WGMMA to compute Q×K^T from already-loaded SMEM tiles
    These run SIMULTANEOUSLY on H100's separate pipelines.
    HBM load latency completely hidden by WGMMA computation.

  Innovation 2: FP8 Support
    Q×K^T computed in FP8 → ~2× faster WGMMA vs BF16
    Online rowwise descaling for numerical stability (no quality loss)

  Performance numbers (H100 SXM5, BF16, S=8192, D=128):
    FA v1: ~350 TFLOPS (35% of peak 989 TFLOPS)
    FA v2: ~544 TFLOPS (55% of peak)
    FA v3: ~741 TFLOPS (75% of peak)
```

```
  Speedup vs standard attention (varying S):
    S=512:   1.9× faster
    S=2048:  3.3× faster
    S=8192:  5.7× faster (O(S²) HBM cost grows; FA stays O(S))
    S=32768: 8.2× faster (FA increasingly dominant)
```

7.9 The Profiling Methodology — nsys → ncu → fix → verify

This is the operationalization of the profiling discipline from Chapter 1. The order matters. Skipping nsys and going straight to ncu is the most common profiling mistake.

7.9.1 The Four-Step Profiling Loop

```
STEP 1: ESTABLISH SYSTEM BASELINE (nsys — 5 minutes)

  Command:
    nsys profile \
      --trace=cuda,nvtx,nccl,osrt \
      --output=profile_%p \
      python workload.py

  What to look for in the timeline:
    Large gaps between GPU kernels → CPU/overhead bottleneck
    NCCL kernels not overlapping with compute → AllReduce not pipelined
    Repeated idle periods → scheduling stall or data loading stall
    Dense, back-to-back GPU kernels → healthy compute utilization

  Decisions from nsys:
    GPU idle > 20% between kernels → overhead-bound (fix: CUDA Graphs,
torch.compile)
    AllReduce > 15% of step time → communication bottleneck (fix: gradient
bucketing)
    Profiler shows correct kernels (FAv3 not FAv1?) → verify kernel selection

  RULE: Only proceed to ncu if nsys confirms a specific kernel is the bottleneck.
        Opening ncu without a target kernel = 100-1000× runtime slowdown and
noise.

STEP 2: KERNEL-LEVEL ANALYSIS (ncu — targeted, 15-30 minutes)

  Command (full profiling of specific kernel):
```

```
    ncu \
      --set full \
      --section SpeedOfLight_RooflineChart \
      --section MemoryWorkloadAnalysis \
      --section ComputeWorkloadAnalysis \
      --section WarpStateStatistics \
      --kernel-name "flash_attention_fwd_kernel" \
      --launch-skip 10 \     # Skip warmup
      --launch-count 1 \     # Profile one launch only
      -o kernel_profile \
      python workload.py

  Key metrics to check:
    METRIC                                    TARGET     RED FLAG
    ──────────────────────────────────────────────────────────────────────
    sm__pipe_tensor_cycles_active (TC util)   > 60%      < 30% → not using TC
    dram__bytes_read (HBM reads)              Near min.  > 3× min → uncoalesced
    smsp__warp_issue_stall_long_scoreboard    < 20 cyc   > 40 → HBM latency
    smsp__warp_issue_stall_barrier            < 5 cyc    > 20 → __syncthreads
    launch__occupancy_limit_registers         False      True → reg spill risk
    sm__throughput.avg.pct_of_peak            > 70%      < 40% → significant gap
    ──────────────────────────────────────────────────────────────────────


STEP 3: APPLY THE CORRECT FIX

  Based on ncu findings, apply in priority order:
    1. Uncoalesced access → fix stride pattern or SMEM transpose
    2. Misaligned shapes → pad to nearest TC-multiple dimension
    3. Low TC utilization at correct dtype → check alignment and tile shapes
    4. Register spills → use __launch_bounds__, reduce variables, recompute
    5. Bank conflicts → add padding column to SMEM arrays
    6. HBM stall warp states → increase tile size (more compute per load)

STEP 4: VALIDATE AT SYSTEM LEVEL (back to nsys)

  A kernel improvement that does NOT move system MFU or TPOT is incomplete.
  It may have exposed the next bottleneck.

  Validation checklist:
    MFU increased (training) OR TPOT decreased (inference)? → Fix applied
    New bottleneck visible in nsys? → Identify next optimization
    No regression in other metrics? → Ship

  The optimization loop is complete when:
```

```
    System metric (MFU/TPOT/throughput) is at target, OR
    The remaining gap is below the hardware's theoretical limit for this workload.
```

7.9.2 Essential Nsight Compute Metrics Reference

```
KEY NCU METRICS FOR AI KERNEL OPTIMIZATION:

COMPUTE:
  sm__pipe_tensor_cycles_active.avg.pct_of_peak    TC utilization %
  sm__throughput.avg.pct_of_peak_sustained_elapsed Overall SM throughput
  sm__inst_executed_pipe_tensor.avg.per_cycle_active TC instructions/cycle

  Target: TC util > 60% for GEMM, > 50% for attention
  Red flag: TC util < 30% at correct dtype → alignment or dtype issue

MEMORY:
  dram__bytes_read.avg                       HBM bytes read
  dram__bytes_write.avg                      HBM bytes written
  lts__t_sectors_op_read.avg.pct_of_peak   L2 read utilization
  l1tex__t_bytes_pipe_lsu_mem_global_op_ld  L1 global load bytes

  Target: HBM reads ≈ theoretical minimum (model size for weight reads)
  Red flag: HBM reads > 3× theoretical → uncoalesced or unnecessary reads

WARP STALL ANALYSIS:
  smsp__warp_issue_stall_long_scoreboard_per_warp_active  Stall on HBM/L2
  smsp__warp_issue_stall_mio_throttle_per_warp_active     Memory queue full
  smsp__warp_issue_stall_barrier_per_warp_active          __syncthreads
  smsp__warp_issue_stall_not_selected_per_warp_active     Scheduler-level

  Dominant stall type → tells you exactly what to fix:
    long_scoreboard dominant → increase tile size (more AI)
    barrier dominant → reduce __syncthreads frequency
    not_selected dominant → not enough warps (increase occupancy)

OCCUPANCY:
  sm__warps_active.avg.pct_of_peak_sustained_active  Achieved occupancy
  launch__occupancy_limit_registers                  Register limiter
  launch__occupancy_limit_shared_memory              SMEM limiter
  launch__occupancy_limit_blocks                     Block count limiter
```

7.9.3 DCGM Counter Diagnostic Table

```
DCGM COUNTER INTERPRETATION FOR KERNEL HEALTH:

  SM_ACTIVE | TENSOR_ACTIVE | DRAM_ACTIVE | Diagnosis
  ──────────────────────────────────────────────────────────────────
  High       High            Low          HEALTHY: compute-bound (training GEMM)
  High       High            High         HEALTHY: bandwidth-bound decode
(correct!)
  High       Low             Low          ⚠ Tensor Cores unused: wrong dtype or
alignment
  High       Low             High         ⚠ Memory-bound elementwise: fusion
target
  Low        Low             Low          × Launch overhead / data stall: CUDA
Graphs
  Low        Low             High         × Bandwidth waste + poor compute: small
kernels
  Variable   Any             Any          × Straggler: one GPU deviating from
cluster
  ──────────────────────────────────────────────────────────────────

  DCGM monitoring command:
    dcgmi dmon -e 1003,1004,1005 -d 1
    (SM_ACTIVE=1003, TENSOR_ACTIVE=1004, DRAM_ACTIVE=1005)
```

7.10 Key Takeaways and Review Questions

```
CHAPTER 7 CORE PRINCIPLES:

  1. GEMM IS 85% OF EVERYTHING.
     For LLM inference and training, ~85% of compute time is in GEMM kernels.
     Optimize GEMM → improve everything.
     Never spend significant time on a kernel that is <5% of step time.

  2. TENSOR CORE ALIGNMENT IS THE SILENT KILLER.
     Misaligned matrix dimensions (not divisible by 16 for BF16, 128 for FP8)
     silently drop from TC throughput to CUDA core throughput: 8× slowdown.
     Check every GEMM shape. Pad non-aligned dimensions.

  3. FLASHATTENTION IS NOT AN OPTIMIZATION — IT IS A PREREQUISITE.
     Standard attention is O(S²) in HBM traffic.
```

```
      FlashAttention is O(S). At S=4K: 67× less HBM traffic.
      At S=128K: standard attention cannot run. FlashAttention can.
      Verify FAv3 is active for H100; verify it is not accidentally using v1/v2.

   4. CUDA GRAPHS REDUCE DISPATCH OVERHEAD 20-100×.
      CPU dispatch overhead is 2-10ms per decode step.
      CUDA Graphs reduce this to 0.01-0.1ms.
      For batch=1 decode (TPOT=30ms): this is 20-30% latency reduction.
      Enable CUDA Graphs for all production serving paths.

   5. LOW OCCUPANCY CAN BE CORRECT.
      FlashAttention: ~12-20% occupancy, achieves 75% of peak TFLOPS.
      Standard attention: ~80% occupancy, achieves 5-15% of peak TFLOPS.
      Occupancy hides memory latency; Tensor Core efficiency uses compute.
      Match the strategy to the bottleneck, not to occupancy targets.

   6. THE PROFILING ORDER IS NON-NEGOTIABLE.
      Always: nsys (system) → confirm kernel is the bottleneck → ncu (kernel).
      Never: open ncu without nsys confirmation.
      Reason: ncu slows execution 100-1000×. Applying it to the wrong kernel
      wastes hours. nsys costs 5 minutes and points you to the right target.

PROFILING COMMAND QUICK REFERENCE:
  "Where does time go?"        nsys profile --trace=cuda,nvtx
  "Why is THIS kernel slow?"   ncu --set full --kernel-name X
  "Is GPU healthy (fleet)?"    dcgmi dmon -e 1003,1004,1005,1009
  "Is AllReduce overlapped?"   nsys with NCCL trace
  "Is it memory or compute?"   ncu Roofline tab: dot position
```

Review Questions

Difficulty: Foundational

Question 1 (Occupancy and Performance): A custom attention kernel reports 82% occupancy in Nsight Compute. A colleague says "82% occupancy means we're using 82% of the GPU." (a) Explain why this statement is incorrect. (b) FlashAttention achieves 18% occupancy but 75% of peak TFLOPS on H100. A naive attention implementation achieves 85% occupancy but only 12% of peak TFLOPS. What explains this counterintuitive result? (c) For a memory-bandwidth-bound kernel, what occupancy level is necessary and sufficient to fully hide H100's HBM latency?

Expected answer: (a) Occupancy measures the fraction of *warp slots* that are active, not actual hardware utilization or efficiency. A warp can be "active" (occupying a slot) while waiting 400 cycles for HBM data. 82% occupancy means 82% of warp slots are allocated — it says nothing

about whether the warps are executing useful work or stalling. (b) FlashAttention's low occupancy is caused by large SMEM usage (~80-100 KB per block). However, it eliminates 67× the HBM traffic vs standard attention — so its SMs spend nearly all their time computing (Tensor Cores active), not waiting for memory. The 18% occupancy is sufficient because the large tile size gives ~200-250 FLOP/byte AI — above the ridge point. Standard attention at 85% occupancy has high occupancy but most warps are stalling on the massive $O(S^2)$ HBM reads — Tensor Cores mostly idle. (c) For a memory-bound kernel with HBM latency ~400 cycles and ~10 instructions between stalls: minimum warps = ceil(400/10) = 40 warps = 40/64 = 62.5% occupancy. In practice: ⩾50% occupancy provides adequate latency hiding for most memory-bound LLM kernels.

Difficulty: Intermediate

Question 2 (FlashAttention IO Analysis): Compute the HBM traffic for standard attention vs FlashAttention for LLaMA-3 70B, for ONE layer, ALL heads, with sequence length S=32,768 and D=128 (head dim), BF16. (a) Standard attention HBM traffic. (b) FlashAttention HBM traffic. (c) If H100 HBM bandwidth is 3.35 TB/s, what is the minimum time each approach requires for attention at this sequence length? (d) Why is standard attention not just slower but completely infeasible at S=128K?

Expected answer: (a) Per head per layer: $4 \times S^2 \times 4B + 4 \times S \times D \times 2B = 4 \times 32768^2 \times 4 + 4 \times 32768 \times 128 \times 2$ = 17.2 GB + 33.6 MB ≈ 17.2 GB. 64 heads × 80 layers = 87.8 TB per forward pass. At 3.35 TB/s: 26.2 seconds minimum. (b) Per head: $4 \times S \times D \times 2B = 4 \times 32768 \times 128 \times 2$ = 33.6 MB. 64 heads × 80 layers = 171.8 GB. At 3.35 TB/s: 51.3 ms minimum. (c) Standard: 26.2 seconds/forward pass. FlashAttention: 51.3 ms. Ratio: 511×. (d) At S=128K: standard attention per-head = $4 \times 128000^2 \times 4$ = 262 GB per head. Total (64 heads × 80 layers) = 1.34 PB — 16,780× more than H100's 80 GB HBM. The attention matrix for even ONE head at ONE layer (262 GB) doesn't fit in H100's entire HBM. Standard attention is physically impossible, not just slow. FlashAttention at S=128K: $4 \times 128000 \times 128 \times 2$ = 131 MB per head, 67.4 GB total — fits, and completes in ~20 ms.

Difficulty: Principal Level

Question 3 (Profiling Diagnosis): You profile a LLaMA-3 70B training step on 8×H100. MFU=38% (target: 55%). Nsight Systems shows: 72% of step time in back-to-back GPU kernels, 28% in NCCL AllReduce. Within the GPU-bound 72%: Nsight Compute shows QKV GEMM achieves 420 TFLOPS (vs 989 TFLOPS peak), with TC utilization at 28%. (a) What is the primary bottleneck causing 38% MFU? (b) What is causing the QKV GEMM to achieve only 420 TFLOPS with 28% TC utilization? (c) Prescribe the exact fix for each issue, in order of impact.

Expected answer: (a) NCCL AllReduce at 28% of step time is the primary MFU bottleneck. With TP=8 over NVLink, AllReduce should be 5-10% of step time at most. 28% suggests: (1) AllReduce

is not overlapped with backward compute — gradient bucketing is not configured, OR (2) TP is crossing InfiniBand boundary instead of staying within NVLink (would cause 10× latency increase per AllReduce), OR (3) gradient bucket size is misconfigured (buckets too small → too many small AllReduces). Fix first: enable gradient_as_bucket_view and check NCCL overlap via nsys timeline. If TP is crossing IB: constrain TP=8 per node. (b) 420 TFLOPS / 989 TFLOPS = 42.5% of peak with 28% TC utilization. This is a TC alignment problem. The QKV projection matrix for LLaMA-3 70B is [d_model × (H_q×d_head + 2×H_kv×d_head)] = [8192 × (64×128 + 2×8×128)] = [8192 × 10240]. 10240 / 128 = 80 (divisible by 128 for FP8 alignment ✓). But if running BF16: 10240 / 256 = 40 ✓. Actually for TP=8: the matrix is sharded → each GPU sees [8192 × 1280]. 1280 / 256 = 5 ✓. This looks aligned — need to check actual shapes with ncu. More likely: FP8 Transformer Engine is not enabled. Without TE: BF16 only = 50% of FP8 throughput = expected ~42% of FP8 peak. (c) In priority order: (1) Enable FP8 via Transformer Engine (`--fp8-e4m3` in Megatron-LM or `fp8_autocast()` in PyTorch TE). Expected gain: 2× GEMM throughput → MFU from 38% to potentially 55%+. (2) Fix AllReduce overlap: `torch.distributed.GradBucket` with `bucket_cap_mb=25`. Verify via nsys that NCCL timeline overlaps backward GEMM. Expected gain: AllReduce from 28% → <10% of step time. Combined: should reach 55-60% MFU.

Difficulty: Principal Level

Question 4 (Custom Kernel Decision): A team is building an LLM serving system. Their profiling shows: QKV GEMM (35% of decode step time), FlashAttention v2 (22%), RMSNorm (0.3%), SiLU activation (0.1%), sampling (0.5%). They are considering writing a custom Triton kernel for RMSNorm. Evaluate this proposal and prescribe the correct optimization portfolio.

Expected answer: The RMSNorm proposal is incorrect prioritization. At 0.3% of step time, even a perfect 10× speedup on RMSNorm saves 0.3% × 0.9 = 0.27% total improvement — entirely unmeasurable. Amdahl's Law: focus only on what is large. Correct optimization portfolio in priority order: (1) Verify FlashAttention v3 is active, not v2. FAv3 on H100 is 34% faster than FAv2. At 22% of step time: 22% × 0.34 = 7.5% total improvement. This is a config change (update vLLM/TRT-LLM to latest version). (2) Enable FP8 via Transformer Engine. GEMM at 35% of step time: 2× faster GEMM → step time decreases 35% × 0.5 = 17.5% total improvement. (3) Enable CUDA Graphs for the full decode path. CPU dispatch overhead (not listed above) is typically 15-25% of actual TPOT — CUDA Graphs capture this. (4) IF RMSNorm + GEMM fusion is not already active: fuse RMSNorm into the adjacent QKV GEMM via torch.compile or Triton fusion. This saves 2 HBM round-trips per norm — at 0.3% of step time, the gain is marginal but it may also reduce memory bandwidth pressure that affects GEMM throughput. At no point is a standalone RMSNorm custom kernel the right answer when the operation is 0.3% of step time.

Further Reading

1. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* — Dao et al. (2022). `arxiv.org/abs/2205.14135` The original FlashAttention paper. The IO complexity proof and SMEM tiling derivation are required reading for any kernel engineer.
2. *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision* — Shah et al. (2024). `arxiv.org/abs/2407.08608` The H100-specific WGMMA + TMA design. The producer-consumer pipeline section demonstrates the state of the art in hardware-software co-design.
3. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations* — Tillet et al. (2019). The foundational Triton paper. `triton-lang.org/main/programming-guide/chapter-1/introduction.html`
4. *CUTLASS: Fast Linear Algebra in CUDA C++* — NVIDIA (2023). `github.com/NVIDIA/cutlass` Read the design document. CUTLASS shows how GEMM tiling maps to hardware.
5. *GPU-Puzzles* — srush (2022). `github.com/srush/GPU-Puzzles` 14 hands-on puzzles that build Triton/CUDA intuition from scratch. Completing all 14 is worth more than reading this chapter twice.
6. *Making Deep Learning Go Brrr From First Principles* — Horace He (2022). `horace.io/brrr_intro.html` The three bottleneck types applied to real PyTorch models. Essential complement to the theory in this chapter.
7. *NVIDIA Nsight Compute Documentation* — NVIDIA (2024). `docs.nvidia.com/nsight-compute/` Learn the metrics, not just the UI. The "Roofline" and "Memory Workload Analysis" sections are the most important for AI workloads.

— End of Chapter 7 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved